

Pandas Programs

1. Handling Missing Data

Aim: To handle missing values in a Pandas DataFrame using fillna() and dropna().

Algorithm:

1. Import pandas and numpy.
2. Create a DataFrame with missing values.
3. Display missing value count.
4. Replace missing Age values with the median.
5. Remove rows with missing Salary.
6. Display the cleaned DataFrame.

Code:

```
import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, np.nan, 30, 22],
    'Salary': [50000, 60000, np.nan, 45000]
}

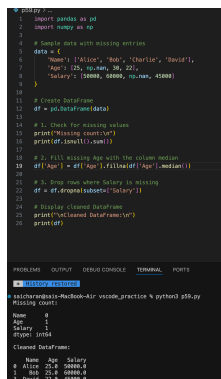
df = pd.DataFrame(data)

print("Missing count:\n")
print(df.isnull().sum())

df['Age'] = df['Age'].fillna(df['Age'].median())
df = df.dropna(subset=['Salary'])

print("\nCleaned DataFrame:\n")
print(df)
```

Output:



```
import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, np.nan, 30, 22],
    'Salary': [50000, 60000, np.nan, 45000]
}

df = pd.DataFrame(data)

print("Missing count:\n")
print(df.isnull().sum())

df['Age'] = df['Age'].fillna(df['Age'].median())
df = df.dropna(subset=['Salary'])

print("\nCleaned DataFrame:\n")
print(df)
```

Missing count:

	Age	Salary
Age	0	1
Salary	1	0

Cleaned DataFrame:

	Name	Age	Salary
0	Alice	25.0	50000.0
1	Bob	22.0	60000.0
3	David	22.0	45000.0

Result: The missing values were successfully handled using fillna() and dropna().

2. Filtering Rows Based on Multiple Conditions

Aim: To filter rows using multiple conditions.

Algorithm:

1. Import pandas.
2. Create DataFrame.
3. Apply multiple conditions using & operator.
4. Display filtered rows.

Code:

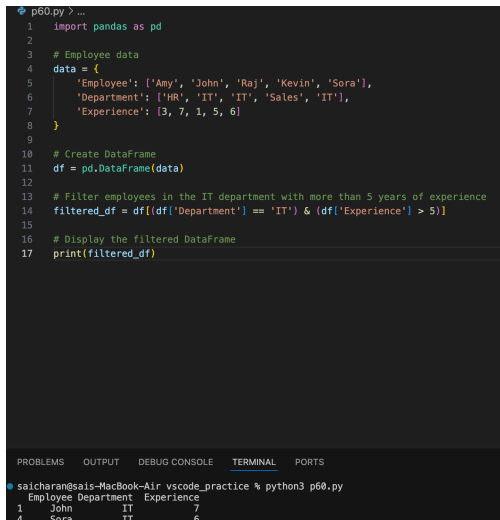
```
import pandas as pd

data = {
    'Employee': ['Amy', 'John', 'Raj', 'Kevin', 'Sora'],
    'Department': ['HR', 'IT', 'IT', 'Sales', 'IT'],
    'Experience': [3, 7, 1, 5, 6]
}

df = pd.DataFrame(data)

filtered_df = df[(df['Department']=='IT') & (df['Experience']>5)]
print(filtered_df)
```

Output:



The screenshot shows a VS Code editor with a Python file named p60.py. The code defines a dictionary 'data' with employee names, departments, and experience levels. It then creates a DataFrame 'df' from this data. A filter is applied to select rows where the department is 'IT' and experience is greater than 5. The filtered DataFrame is printed, showing two rows: John (IT, 7) and Sora (IT, 6).

```
p60.py > ...
1 import pandas as pd
2
3 # Employee data
4 data = {
5     'Employee': ['Amy', 'John', 'Raj', 'Kevin', 'Sora'],
6     'Department': ['HR', 'IT', 'IT', 'Sales', 'IT'],
7     'Experience': [3, 7, 1, 5, 6]
8 }
9
10 # Create DataFrame
11 df = pd.DataFrame(data)
12
13 # Filter employees in the IT department with more than 5 years of experience
14 filtered_df = df[(df['Department'] == 'IT') & (df['Experience'] > 5)]
15
16 # Display the filtered DataFrame
17 print(filtered_df)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@saais-MacBook-Air: % python3 p60.py
Employee Department Experience
1 John IT 7
4 Sora IT 6
```

Result: The required records were filtered successfully.

3. Aggregating and Grouping Data

Aim: To group data and calculate aggregate values.

Algorithm:

1. Import pandas.
2. Create DataFrame.
3. Group by Store.
4. Calculate total Sales and average Employees.
5. Display summary.

Code:

```
import pandas as pd

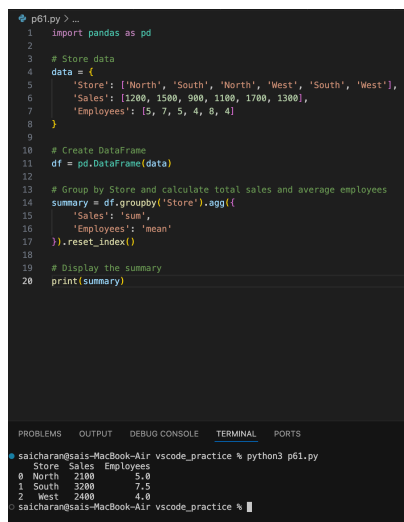
data = {
    'Store': ['North', 'South', 'North', 'West', 'South', 'West'],
    'Sales': [1200, 1500, 900, 1100, 1700, 1300],
    'Employees': [5, 7, 5, 4, 8, 4]
}

df = pd.DataFrame(data)

summary = df.groupby('Store').agg({
    'Sales': 'sum',
    'Employees': 'mean'
}).reset_index()

print(summary)
```

Output:



```
p61.py -
1 import pandas as pd
2
3 # Store data
4 data = {
5     'Store': ['North', 'South', 'North', 'West', 'South', 'West'],
6     'Sales': [1200, 1500, 900, 1100, 1700, 1300],
7     'Employees': [5, 7, 5, 4, 8, 4]
8 }
9
10 # Create DataFrame
11 df = pd.DataFrame(data)
12
13 # Group by Store and calculate total sales and average employees
14 summary = df.groupby('Store').agg({
15     'Sales': 'sum',
16     'Employees': 'mean'
17 }).reset_index()
18
19 # Display the summary
20 print(summary)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air vscode_practice % python3 p61.py
Store Sales Employees
0 North 2100 5.0
1 South 3200 7.5
2 West 2400 4.0
saicharan@sais-MacBook-Air vscode_practice %
```

Result: The data was grouped and aggregated successfully.

4. Python Program Using describe()

Aim: To generate statistical summaries using describe().

Algorithm:

1. Import pandas.
2. Create DataFrame.
3. Display numeric summary.
4. Display categorical summary.

Code:

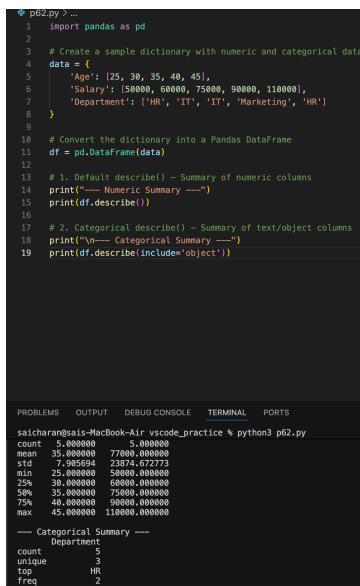
```
import pandas as pd

data = {
    'Age': [25, 30, 35, 40, 45],
    'Salary': [50000, 60000, 75000, 90000, 110000],
    'Department': ['HR', 'IT', 'IT', 'Marketing', 'HR']
}

df = pd.DataFrame(data)

print(df.describe())
print(df.describe(include='object'))
```

Output:

The screenshot shows a Python IDE with a dark theme. The top pane contains the Python code for creating a DataFrame and using the describe() function. The bottom pane shows the output of the code, which includes a numeric summary and a categorical summary. The numeric summary shows statistics for the 'Age' and 'Salary' columns, while the categorical summary shows the distribution of the 'Department' column.

```
p62.py ?...
1 import pandas as pd
2
3 # Create a sample dictionary with numeric and categorical data
4 data = {
5     'Age': [25, 30, 35, 40, 45],
6     'Salary': [50000, 60000, 75000, 90000, 110000],
7     'Department': ['HR', 'IT', 'IT', 'Marketing', 'HR']
8 }
9
10 # Convert the dictionary into a Pandas DataFrame
11 df = pd.DataFrame(data)
12
13 # 1. Default describe() - Summary of numeric columns
14 print("---- Numeric Summary ----")
15 print(df.describe())
16
17 # 2. Categorical describe() - Summary of text/object columns
18 print("\n--- Categorical Summary ---")
19 print(df.describe(include='object'))
```

	Age	Salary
count	5.000000	5.000000
mean	35.000000	77000.000000
std	7.285504	23874.672773
min	25.000000	50000.000000
25%	30.000000	60000.000000
50%	35.000000	75000.000000
75%	40.000000	90000.000000
max	45.000000	110000.000000

	Department
count	5
unique	3
top	HR
freq	2

Result: The describe() function generated statistical summaries successfully.

5. head() and tail() in Python

Aim: To display the first and last rows of a DataFrame.

Algorithm:

1. Import pandas.
2. Create DataFrame.
3. Use head().
4. Use tail().
5. Display results.

Code:

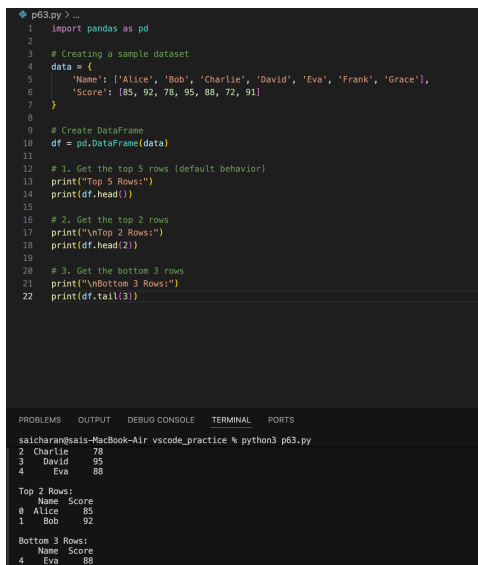
```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace'],
    'Score': [85, 92, 78, 95, 88, 72, 91]
}

df=pd.DataFrame(data)

print(df.head())
print(df.head(2))
print(df.tail(3))
```

Output:



```
p63.py? ...
1 import pandas as pd
2
3 # Creating a sample dataset
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace'],
6     'Score': [85, 92, 78, 95, 88, 72, 91]
7 }
8
9 # Create DataFrame
10 df = pd.DataFrame(data)
11
12 # 1. Get the top 5 rows (default behavior)
13 print("Top 5 Rows:")
14 print(df.head())
15
16 # 2. Get the top 2 rows
17 print("\nTop 2 Rows:")
18 print(df.head(2))
19
20 # 3. Get the bottom 3 rows
21 print("\nBottom 3 Rows:")
22 print(df.tail(3))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air: vscode_practice % python3 p63.py
2 Charlie 78
3 David 95
4 Eva 88

Top 2 Rows:
   Name  Score
0  Alice    85
1   Bob    92

Bottom 3 Rows:
   Name  Score
4   Eva    88
5  Frank    72
6  Grace    91
```

Result: The head() and tail() functions displayed the required rows successfully.